RegRadar: Autonomous, Zero-Destruction Compliance Remediation for Infrastructure-as-Code

ABSTRACT

Modern cloud governance relies heavily on Infrastructure-as-Code (IaC), yet reconciling complex regulatory mandates (such as HIPAA and GDPR) with declarative configurations remains slow and error-prone. Existing static analysis tools detect syntax violations but lack semantic context and automated remediation, while unconstrained Large Language Model (LLM) agents introduce hallucinations, non-deterministic outputs, and state-destructive infrastructure mutations.

This project presents RegRadar, a neuro-symbolic cloud governance engine that unifies deterministic program analysis with multi-agent intelligence for autonomous compliance remediation. RegRadar parses IaC configurations into an Abstract Syntax Tree (AST) to construct a directed acyclic resource dependency graph. By calculating graph centrality metrics, the engine algorithmically bounds blast-radius risk before triggering remediation. A coordinated agent swarm, guided by ontological regulatory retrieval (GraphRAG), synthesizes context-aware code patches. Crucially, proposed patches undergo sandboxed dry-run validation to enforce zero-destruction runtime invariants, rejecting any modification that triggers unintended state deletion or service replacement. Evaluated against deliberately vulnerable infrastructure benchmarks, RegRadar eliminates configuration drift, guarantees state-safe automated repairs, and bridges the gap between regulatory intent and resilient cloud infrastructure.

REVIEW

1. Empirical Fault Analysis and Runtime Failure in Declarative IaC

A central premise of RegRadar is that unvalidated modifications to declarative infrastructure configurations often result in deployment failures and runtime state corruption. In a comprehensive empirical investigation across Infrastructure-as-Code (IaC) ecosystems, Drosos et al. (2024) analyzed 360 real-world IaC bugs and established that nearly half (45%) stem from flawed logic regarding system interactions and environment state manipulation. Furthermore, 37% of these faults triggered abrupt deployment terminations, underscoring that declarative configurations behave as fully functional software programs requiring rigorous pre-deployment verification rather than unchecked automated updates (Drosos et al., 2024).

2. Limitations of Deterministic Scanners and Security Smell Taxonomies

While deterministic scanners like Checkov are widely deployed, empirical research shows that static checkers alone leave substantial governance gaps. In an empirical study evaluating 812 open-source Terraform repositories, Verdet et al. (2023) evaluated compliance across major cloud providers (AWS, Azure, and GCP) using static analysis and observed persistent gaps in security policy enforcement, highlighting the need for automated mechanisms that actively facilitate remediation.

To address the limitations of technology-specific rule checkers, Saavedra and Ferreira (2022) developed GLITCH, demonstrating that transforming declarative configurations into intermediate representations (IR) across multiple IaC platforms achieves higher precision and recall than siloed, regex-based linters. Expanding on earlier smell categories, War, Nikiema, et al. (2025) introduced an updated taxonomy comprising 62 fine-grained security smells across seven modern IaC technologies, documenting that misconfigurations persist in version control systems for extended lifecycles due to the lack of automated, context-aware remediation frameworks.

3. Automated Program Repair (APR) and Semantic Grounding

Automating repairs for declarative infrastructure specifications requires structural constraint enforcement. Henkel et al. (2021) addressed this by developing Shipwright, an automated repair system for container specifications that leverages Abstract Syntax Tree (AST) analysis and clustering to synthesize valid repair templates while maintaining a human-in-the-loop verification barrier.

Because rule-based systems struggle to evaluate contextual intent, War, Rawass, et al. (2025) demonstrated that augmenting code analysis with language models substantially outperforms traditional keyword matching and superficial syntactic checkers in identifying misconfigurations. Their findings validate the hybrid neuro-symbolic approach: pairing structural code representations with semantic reasoning improves detection accuracy and provides the contextual awareness necessary to synthesize valid configuration repairs (War, Rawass, et al., 2025).